FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

2

RULE – BASED AGENT

Student Name	Gorospe Paolo C
Section:	An43
Professor:	Crisola Tan
Date:	

I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

II. TECHNICAL ASSESSMENT

RULE – BASED AGENT

Objective

At the end of this laboratory activity, students are expected to:

1. Explain the concept of rule-based agents.
2. Implement a rule-based agent in Python.
3. Interact with an environment by perceiving and acting based on rules.
4. Observe and analyze the behavior of an agent in a controlled environment.

Materials Needed

- Computer with Python 3.x installed
- Text editor or IDE (e.g., VSCode, PyCharm)
- Optional: Jupyter Notebook for interactive testing

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA2_lastname.pdf**
   (Example: TA2_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Laboratory Task

Pre-Lab Preparation

1. Review the Python concepts:
   - Classes and objects

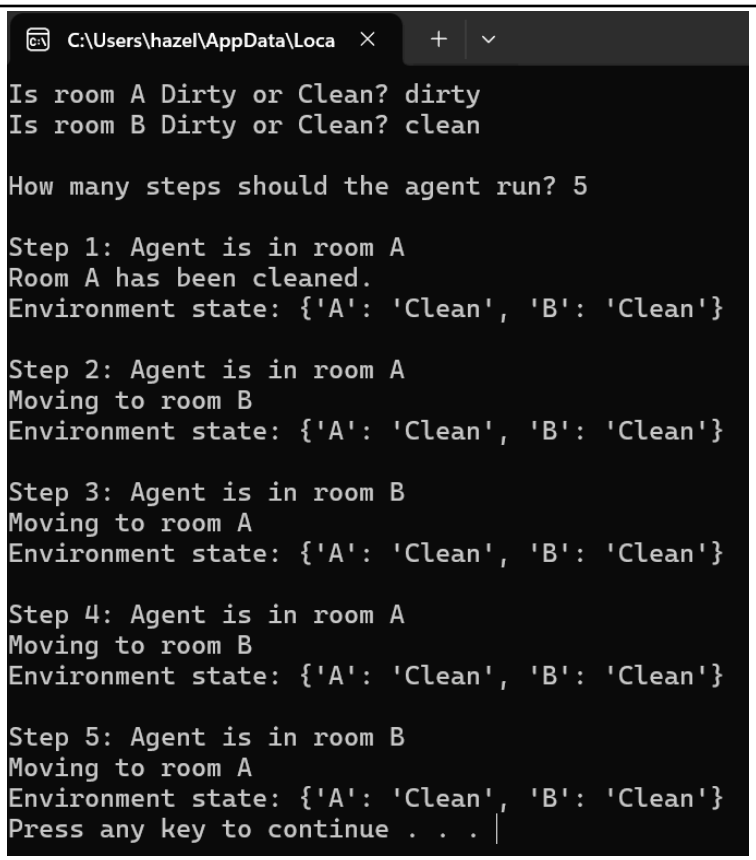
- If-Else statements
- Loops (for loop)
- Functions / methods

2. Understand the environment:
    - Two rooms: A and B
    - Each room can be Dirty or Clean

Task 1: Set Up the Environment

1. Initialize two rooms using a Python dictionary.
2. Allow the user to set the **initial state** of each room (dirty or clean).
3. Write a function to **check if a room is dirty** and a function to **clean a room**.

Sample Output




```
rooms = {
    "A": input("Is room A Dirty or Clean? ").strip().capitalize(),
    "B": input("Is room B Dirty or Clean? ").strip().capitalize()
}

def is_dirty(room):
    return rooms[room] == "Dirty"

def clean_room(room):
    rooms[room] = "Clean"

steps = int(input("\nHow many steps should the agent run? "))
current_room = "A"

for step in range(1, steps + 1):
    print(f"\nStep {step}: Agent is in room {current_room}")
```

```
    if is_dirty(current_room):
        clean_room(current_room)
        print(f"Room {current_room} has been cleaned.")
    else:
        current_room = "B" if current_room == "A" else "A"
        print(f"Moving to room {current_room}")

    print("Environment state:", rooms)
```

```
PS D:\Dev\C# Start>  d:; cd 'd:\Dev\C# Start'; & 'c:\Users\paolo\AppData

Is room A Dirty or Clean? dirty
Is room B Dirty or Clean? clean

How many steps should the agent run? 6

Step 1: Agent is in room A
Room A has been cleaned.
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 2: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 3: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 4: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 5: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 6: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}
PS D:\Dev\C# Start>
```

Task 2: Implement the Rule-Based Agent

1. Create a class for the agent.

2. Implement the **perceive-and-act method**:
    - Rule 1: If the current room is dirty, clean it.
    - Rule 2: If the current room is clean, move to the other room.
3. Implement a **move method** to switch rooms.

```
rooms = {
    "A": input("Is room A Dirty or Clean? ").strip().capitalize(),
    "B": input("Is room B Dirty or Clean? ").strip().capitalize()
}


def is_dirty(room):
    return rooms[room] == "Dirty"


def clean_room(room):
    rooms[room] = "Clean"


class Agent:
    def __init__(self):
        self.current_room = "A"

    def move(self):
        self.current_room = "B" if self.current_room == "A" else "A"
        print(f"Moving to room {self.current_room}")

    def perceive_and_act(self):
        if is_dirty(self.current_room):
            clean_room(self.current_room)
            print(f"Room {self.current_room} has been cleaned.")
        else:
            self.move()


agent = Agent()
steps = int(input("\nHow many steps should the agent run? "))

for step in range(1, steps + 1):
```

```
    print(f"\nStep {step}: Agent is in room {agent.current_room}")
    agent.perceive_and_act()
    print("Environment state:", rooms)
```

```
PS D:\Dev\C# Start> & C:/Users/paolo/AppData/Local/Python/bin
Is room A Dirty or Clean? clean
Is room B Dirty or Clean? dirty

How many steps should the agent run? 6

Step 1: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Dirty'}

Step 2: Agent is in room B
Room B has been cleaned.
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 3: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 4: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 5: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 6: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}
PS D:\Dev\C# Start>
```

Task 3: Run the Simulation

1. Ask the user how many steps the simulation should run.
2. Run the agent in a loop for the specified steps.
3. After each step, print:
    - The agent's current location
    - The action taken (cleaned/moved)
    - Current state of the environment

```
rooms = {
    "A": input("Is room A Dirty or Clean? ").strip().capitalize(),
    "B": input("Is room B Dirty or Clean? ").strip().capitalize()
}
```

```python
def is_dirty(room):
    return rooms[room] == "Dirty"


def clean_room(room):
    rooms[room] = "Clean"


class Agent:
    def __init__(self):
        self.current_room = "A"

    def move(self):
        self.current_room = "B" if self.current_room == "A" else "A"

    def perceive_and_act(self):
        if is_dirty(self.current_room):
            clean_room(self.current_room)
            return f"Cleaned room {self.current_room}"
        else:
            self.move()
            return f"Moved to room {self.current_room}"


agent = Agent()
steps = int(input("\nHow many steps should the agent run? "))

for step in range(1, steps + 1):
    action = agent.perceive_and_act()

    print(f"\nStep {step}")
    print("Current location:", agent.current_room)
    print("Action:", action)
    print("Environment state:", rooms)
```

```
PS D:\Dev\C# Start> & C:/Users/paolo/AppData/Local/Python/bin/python.exe "d:/Dev/C# St
Is room A Dirty or Clean? clean
Is room B Dirty or Clean? dirty

How many steps should the agent run? 7

Step 1
Current location: B
Action: Moved to room B
Environment state: {'A': 'Clean', 'B': 'Dirty'}

Step 2
Current location: B
Action: Cleaned room B
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 3
Current location: A
Action: Moved to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 4
Current location: B
Action: Moved to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 5
Current location: A
Action: Moved to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 6
Current location: B
Action: Moved to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}

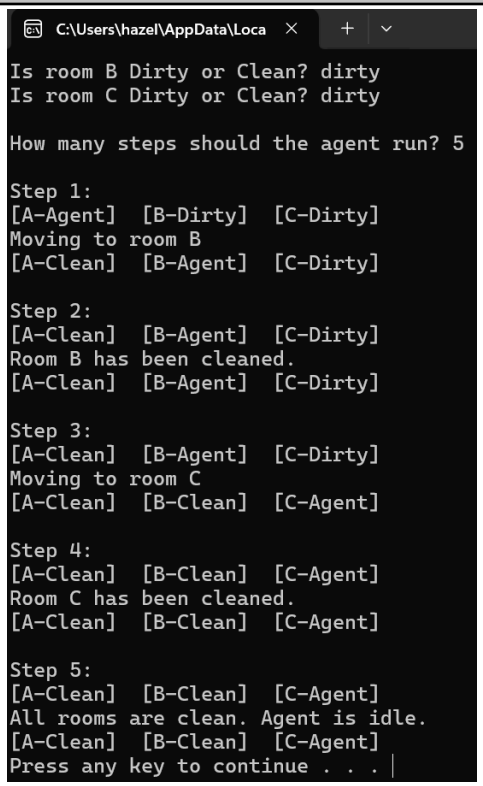
Step 7
Current location: A
Action: Moved to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}
```

Optional (Bonus Task)

- Add a **third room (C)** and update the agent logic.

- Make the agent **randomly choose which dirty room to clean next**.
- Visualize the agent moving between rooms using a **text-based grid**.

Sample Output (Bonus Task)




```python
import random

rooms = {}
for room in ("A", "B", "C"):
    rooms[room] = input(
        f"Is room {room} Dirty or Clean? "
    ).strip().capitalize()


def is_dirty(room):
    return rooms[room] == "Dirty"


def clean_room(room):
    rooms[room] = "Clean"
```

```
class Agent:
    def __init__(self):
        self.location = "A"

    def move(self, room):
        self.location = room

    def perceive_and_act(self):
        if is_dirty(self.location):
            clean_room(self.location)
            return f"Cleaned room {self.location}"

        dirty_rooms = [
            room for room in rooms if is_dirty(room)
        ]
        if dirty_rooms:
            self.move(random.choice(dirty_rooms))
            return f"Moved to room {self.location}"

        return "No action—all rooms are clean"


def show_grid(agent):
    print("+--------------------+")
    for room in rooms:
        marker = "<- Agent" if agent.location == room else ""
        print(f"| {room}: {rooms[room]:5} {marker:8} |")
        print("+--------------------+")


agent = Agent()
steps = int(input("How many steps should the agent run? "))

for step in range(1, steps + 1):
    action = agent.perceive_and_act()
```

```
    print(f"\nStep {step}: {action}")
    show_grid(agent)
```

```
How many steps should the agent run? 6

Step 1: Moved to room B
+--------------------+
| A: Clean           |
+--------------------+
| B: Dirty <- Agent  |
+--------------------+
| C: Clean           |
+--------------------+

Step 2: Cleaned room B
+--------------------+
| A: Clean           |
+--------------------+
| B: Clean <- Agent  |
+--------------------+
| C: Clean           |
+--------------------+

Step 3: No action—all rooms are clean
+--------------------+
| A: Clean           |
+--------------------+
| B: Clean <- Agent  |
+--------------------+
| C: Clean           |
+--------------------+

Step 4: No action—all rooms are clean
+--------------------+
| A: Clean           |
+--------------------+
| B: Clean <- Agent  |
+--------------------+
| C: Clean           |
+--------------------+

Step 5: No action—all rooms are clean
+--------------------+
| A: Clean           |
+--------------------+
| B: Clean <- Agent  |
+--------------------+
| C: Clean           |
+--------------------+

Step 6: No action—all rooms are clean
+--------------------+
| A: Clean           |
+--------------------+
| B: Clean <- Agent  |
+--------------------+
| C: Clean           |
+--------------------+
```

Your Code - Laboratory Tasks

Task 1:

```
rooms = {
    "A": input("Is room A Dirty or Clean? ").strip().capitalize(),
    "B": input("Is room B Dirty or Clean? ").strip().capitalize()
}

def is_dirty(room):
    return rooms[room] == "Dirty"

def clean_room(room):
    rooms[room] = "Clean"

steps = int(input("\nHow many steps should the agent run? "))
current_room = "A"

for step in range(1, steps + 1):
    print(f"\nStep {step}: Agent is in room {current_room}")

    if is_dirty(current_room):
        clean_room(current_room)
        print(f"Room {current_room} has been cleaned.")
    else:
        current_room = "B" if current_room == "A" else "A"
        print(f"Moving to room {current_room}")

    print("Environment state:", rooms)
```

Task 2:

```
rooms = {
    "A": input("Is room A Dirty or Clean? ").strip().capitalize(),
    "B": input("Is room B Dirty or Clean? ").strip().capitalize()
}


def is_dirty(room):
    return rooms[room] == "Dirty"
```

```
def clean_room(room):
    rooms[room] = "Clean"


class Agent:
    def __init__(self):
        self.current_room = "A"

    def move(self):
        self.current_room = "B" if self.current_room == "A" else "A"
        print(f"Moving to room {self.current_room}")

    def perceive_and_act(self):
        if is_dirty(self.current_room):
            clean_room(self.current_room)
            print(f"Room {self.current_room} has been cleaned.")
        else:
            self.move()


agent = Agent()
steps = int(input("\nHow many steps should the agent run? "))

for step in range(1, steps + 1):
    print(f"\nStep {step}: Agent is in room {agent.current_room}")
    agent.perceive_and_act()
    print("Environment state:", rooms)
```

Task 3:

```
rooms = {
    "A": input("Is room A Dirty or Clean? ").strip().capitalize(),
    "B": input("Is room B Dirty or Clean? ").strip().capitalize()
}


def is_dirty(room):
    return rooms[room] == "Dirty"
```

```
def clean_room(room):
    rooms[room] = "Clean"


class Agent:
    def __init__(self):
        self.current_room = "A"

    def move(self):
        self.current_room = "B" if self.current_room == "A" else "A"

    def perceive_and_act(self):
        if is_dirty(self.current_room):
            clean_room(self.current_room)
            return f"Cleaned room {self.current_room}"
        else:
            self.move()
            return f"Moved to room {self.current_room}"


agent = Agent()
steps = int(input("\nHow many steps should the agent run? "))

for step in range(1, steps + 1):
    action = agent.perceive_and_act()

    print(f"\nStep {step}")
    print("Current location:", agent.current_room)
    print("Action:", action)
    print("Environment state:", rooms)
```

Optional:

```
import random
                    ⬇
rooms = {}
for room in ("A", "B", "C"):
```

```
    rooms[room] = input(
        f"Is room {room} Dirty or Clean? "
    ).strip().capitalize()


def is_dirty(room):
    return rooms[room] == "Dirty"


def clean_room(room):
    rooms[room] = "Clean"


class Agent:
    def __init__(self):
        self.location = "A"

    def move(self, room):
        self.location = room

    def perceive_and_act(self):
        if is_dirty(self.location):
            clean_room(self.location)
            return f"Cleaned room {self.location}"

        dirty_rooms = [
            room for room in rooms if is_dirty(room)
        ]
        if dirty_rooms:
            self.move(random.choice(dirty_rooms))
            return f"Moved to room {self.location}"

        return "No action—all rooms are clean"


def show_grid(agent):
    print("+--------------------+")
```

```
    for room in rooms:
        marker = "<- Agent" if agent.location == room else ""
        print(f"| {room}: {rooms[room]:5} {marker:8} |")
        print("+--------------------+")


agent = Agent()
steps = int(input("How many steps should the agent run? "))

for step in range(1, steps + 1):
    action = agent.perceive_and_act()
    print(f"\nStep {step}: {action}")
    show_grid(agent)
```

Task 4: Analysis Questions

After running the simulation, answer the following:

1. What rules does the agent follow?
2. How does the agent decide when to clean and when to move?
3. What happens if both rooms are already clean at the start?
4. How would you modify the agent to handle **more than two rooms**?

Answer the Questions ⇩

1. The agent follows two rules: clean the current room if it is dirty; otherwise, move to the other room.
2. On each step, it checks the state of its current room with is_dirty(). If the result is true, it cleans the room. If false, it moves. It performs one action per step.
3. If both rooms start clean, the agent moves between A and B on every step. It never cleans a room.
4. Add the extra rooms to the rooms dictionary and change move() to cycle through a list such as ["A", "B", "C"] instead of switching only between A and B.